Review of Lab 3

David Athersych, P.Eng.
October 2021

This is a several-page note to answer what may have seemed like a simple question. Please read it all the way through.

A student wrote:

> *"I am wondering if the labs in the rest of the course can be expected to be as time consuming as this previous lab (referring to lab 3). … Lab 2 seemed like a good challenge and I finished on time, however if it weren't for the 48hr extension to Lab 3 I would not have come close to finishing part 2, and I know a handful of my peers that I spoke to felt the same. In your opinion will this become the norm and as such I should start further in advance?*
>
> *"Any advice would be greatly appreciated, my only experience with programming has been APSC 143 so this course is one of the more foreign concepts to me."*

Lab 3 generated – as I expected – much more question traffic than labs 1 and 2. Lab 3 part 2 required that learners begin to pull together a number of concepts – pointers, linked data structures, input/output, ***how to plan a programming solution*** and ***how to test/debug the code***. This was the first time that there was no outline or template code available – just a suggestion that you could get some ideas from another piece of code provided.

Lab 3 also demonstrated some of the problems we have with the class format in place this year. Learning is usually a social process, meaning that learners interact with instructors to ask questions – in my experience, often before and after classes – and instructors can sense when a part of the material needs more discussion in class before moving on. Lots of times, a question in class led to blackboard/whiteboard time where we got issues with the material cleared up. This year, we try to make do with 5 sessions a week of live contact.

Engineering is a social process as well. Engineering projects may require people from different backgrounds and/or with different levels of expertise – working together to create a solution to a problem. Ordinarily, I would have labs done in groups of 2, and I would fully expect that ideas would be discussed in lounges, study rooms, cafeterias, and coffee shops.

For the safety of everyone, we are stuck with our separated environment, but we can figure out how to cope.

For labs 5 and 6, I am offering the option of working together. I know there are some people who are already in close contact – for those folks, group formation should be easy. For those who need some help finding a partner, I can help, **BUT** I need your authorization to share your name and email address with anyone else. (Under privacy laws and Queen's rules, all your personal information must not be shared without your explicit permission.) So, if you are looking, send me an email and I will post a list on onQ. If you prefer not to spread your information widely, tell me and I will make an arbitrary assignment of a partner – from the list of people who have contacted me.

So, lets talk about the two items I have highlighted near the top of my first paragraph - ***how to plan a programming exercise*** and ***how to test/debug the code***.

How to Solve a Programming Problem

Look back on how you approached Deliverable 2 in Lab 3. Did you start by developing a plan for how you were going to design and build the code? Did you include in your plan some ideas about how you were going to test your code as you developed it?

Do not worry – I really was not expecting "Yes" for an answer. We will use this as a teaching moment.

1.Think About Requirements

After reading the requirement over, you can enumerate those requirements:

1. Must write a verifyBST() routine to confirm that a tree is really a BST.
2. Must be able to keep track of how many nodes are in the tree.
3. Read the IDENTS file and for each one found, put the data in a BST.
4. Read the LOOKUPs file and for each line found, try to find it in the tree.
5. Read the DELETES file and for each file found, try to find it in the tree.

First, what do we know?

1. A verifyBST() routine is just a routine to recursively traverse the tree and make sure that every parent has a left child with a smaller value and a right child with a larger value. We can write this and test it with the code for Deliverable 1. It will need a little tinkering to put it into Deliverable 2, because the tree node is different, but otherwise it is the same idea.
2. Keeping track of node count is easy. We will keep the count in the tree descriptor. We will add 1 to it each time we add a node. We will subtract 1 from it each time we delete a node. Or we could write a routine to compute count for the whole tree and call it before and after a change. Performance-wise, the first plan is better, but there is nothing in the requirement that says performance is important.

3. When we put data into the tree, we are going to want to see if the insert worked. So, we will need a function to print the contents of a tree. We will only use this for testing.
4. We probably do not want to test our code with the instructor's big file. What was he thinking? There must be a million lines in that file! We will have to create some small files to test with, and we will have to think about what should be in those files to do a proper test.
5. What would constitute a proper test for insert?
    a. Insert into empty tree.
    b. Insert to the left subtree of root.
    c. Insert to the right subtree of root.
    d. Insert to the left subtree of root's left subtree.
    e. Insert to the right subtree of root's left subtree.
    f. Insert to the left subtree of root's right subtree.
    g. Insert to the right subtree of root's right subtree.
    h. Try a couple of duplicates and make sure that the expected behavior occurs.
6. We have lots of code for BSTs. We have the code we finished for Deliverable 1. We have the other examples that were provided, including the BST_STRING source file mentioned in the lab sheet.
7. We must figure out how to read those files.

Before we try solving anything, let us review the code we have, with the requirements in mind.

2. Approach to Solving

Once we have spent some time thinking about the requirements and searching through the code we have for parts of a solution, we can start tackling the things we need to solve.

Looking through the list above, we have a good idea how to do 1 – the verifyBST() routine and 2- keeping track of count. Reading the files might be nice to solve, because if we can read data, then we can use that to build the insert (which we already know how to do from the first deliverable).

The IDENT file has two fields – a 10-digit employee number and a password. The field are separated by blanks.
The LOOKUP file has one field – a 10-digit employee number.
The DELETE file has one field – a 10-digit employee number.

Everything needs to be able to read a 10-digit employee number. So, what will be do with it? It would be convenient to be able to work with it as an integer, because all the code we have looked at uses an integer for a key. We want to use employee number as a key. So, everything hinges on getting the first field as an integer.

We have two choices – read the first item in a file using fscanf(), or read a whole line as a string using fgets().

fscanf() approach:

```
n = fscanf (fp, "%d", &employeeid);
```

This will read data from the input file (fp), accepting characters that belong in an integer – optional leading "whitespace" – blanks, newlines, tab characters - an optional leading plus sign + or minus sign – followed by characters 0 through 9 (remember, everything in the file is a bunch of characters) . As soon as a character that does not belong to an integer appears, the reading stops. The non-integer character stays in the file. (Well, not really, but the system behaves as if it does.)

The system converts the set of characters to the integer value that someone reading the string of characters would expect.
We can use this for reading all three files! It is all we need for the LOOKUPs and the DELETEs files.

For the IDENTS file, after reading the number, we want to read the password. First, read all the blanks that separate the fields.

```
do {
     ch = fgetc (fp);    // fetch next character from file
} while (ch == ' ');
```

This loop finishes when we hit the first non-blank. This will be the first character in the password.

We could accumulate the password in a buffer.

```
char     passwordbuffer [20];
int  i=0;

do  {
    passwordbuffer[i] = ch;  // first time, catches
                             // character from above
     i++;
     ch = fgetc (fp);        // get next character
} while (ch != '\r' && ch != '\n');
passwordbuffer[i] = '\0';    // make proper string
```

So, we have the pieces necessary to read the file. Before going any further, incorporate them into a piece of code that reads the components of a line from the file, prints out the employee number and the password. Check to see if it works.

Then use this with your routine to build a tree node. Pay attention to how you copy the password into the node. If you created an array directly in the node, you must do a proper string copy. If you just have a pointer to the string in your node, you must use string duplicate (**`strdup()`**) to make a copy of what is in passwordbuffer – otherwise it will get overwritten by the next line read.

If you have a function to print the contents of a tree node, you can use it to print the nodes as you read a test file with 3 or 4 values. If the displayed values match what you want, you are ready to finish the insert function. If not, you have a little debugging to do. However, when you finish any debugging required, you will then have confidence that you are reading data and building nodes properly.

Oh, what was the other input approach?

fgets() approach:

Another way to handle the input is:

```
p = fgets (pbuff, 256, fp);
// check if read worked - if not, we're done
if (p == NULL)   {
     printf ("--EOF found. Input complete\n");
     break;
     }
if (verbose)    printf ("--Input: %s", p);
parsedata (pbuff, &empid, &ppswd);
printf ("Employee ID: %010d  Password: %s\n", empid, ppswd);
```

That is – get the whole line of data from the file, and call a routine to parse it.

```
int parsedata (char *buffer, int *pemp, char **pppswd)
// Function takes buffer containing an input line and pulls out
// employee number and password. Employee number returned in
// variable pointed to by pemp, pointer to password in the
// supplied buffer (!!) returned in variable pointed to by
// pppwd.
{
     int       rslt = -1;
     if (buffer != NULL && pemp != NULL)     {
          char *p = buffer;        // temporary buffer pointer
          // Get employee id
          *pemp = get_empid (buffer);
          // we know field is 10 digits, but we could have
          // looped skipping digits
          p +=10;
          // skip over blanks before password
          while (*p == ' ')   p++;
          // Save pointer to password in buffer. Actual copy
          // gets made when node is saved.
```

```
          *pppswd = p;
          // Now search for end-of-line and replace with
          // an end-of-string
          p++;
          while (*p != '\n' && *p != '\r') p++;
          *p = '\0';
          rslt = 0;
          }
     return rslt;
}//parsedata()
```

This depends on another routine just to parse an employee id:

```
int get_empid (char *buffer)
// Employee id is right at start of buffer.
{
     int       rslt = -1;
     if (buffer != NULL) {
          rslt = atoi (buffer);
          }
     return rslt;
}//get_empid()
```

Once we have the data, our testing strategy is exactly the same as in the fscanf() approach.

3. Incremental Building

Ok – we can fetch data and build nodes; can we build a BST? Of course, we already have the code in Deliverable 1. It just needs to be tinkered with to make it work with our nodes.
Just implement insert and use the test code you developed to display the tree as built. If it is right, it is time to move on; if not, it is only the insert that needs inspection, because we are confident that the input and the creation of nodes works.

And we have a find as well. We want a find, because it is necessary to do a find before we do a delete. Ok, it too needs some tinkering.

Since we have all that code, there is just some small adjustments to make, a little testing on some small sample data, and then a run on the great big file that came with the lab.

Does that seem a bit easier now that I have walked through it?

Summary

This answer started off as a response to a simple question: **Will the next labs be as hard or harder than lab 3?**

I hope that I have convinced you the answer can be: **No**.

They may involve a similar requirement for coding, but I hope that some of my comments on how to prepare for a lab will help make the task easier.